



Faculty of Engineering & Technology

Electrical & Computer Engineering Department

Operating Systems ENCS3390

Project 1

Prepared by:

Name: Rawand Radi

Number: 1221054

Instructor: Mohammed Khalil

Section: 4

Date: 8/12/2024

Abstract

In this project, we present three approaches for a word frequency count program: Naive, Multiprocessing, and Multi-threading. We evaluate the execution time for each approach and create a comparative table to highlight the best execution time and throughput. The goal is to analyze the performance differences between these methods and identify the most efficient approach for large-scale word frequency counting tasks.

Table of Content

Abstract.....	I
Table of Content.....	II
Table of Figures.....	III
Theory.....	4
Naive approach.....	4
multiprocessing approach.....	4
Multi-threading approach.....	5
Discussion and code analysis.....	6
Common Code analysis.....	6
Naive approach.....	7
Multiprocessing Approach.....	9
Multi-threading approach.....	13
Comparing performance of the approaches.....	18
An analysis according to Amdahl's law.....	19
Questions analysis.....	20
Work environment.....	22
Conclusion.....	23
References.....	24

Table of Figures

Figure 1 :Read from file function.	6
Figure 2 : Word-Freq struct.	6
Figure 3 : snippet code of the Naive.	7
Figure 4 : Naive results.	7
Figure 5 : Naive execution time.	8
Figure 6 : snippet code of multiprocessing.	9
Figure 7 :dividing to chunks code.	9
Figure 8 : shared memory creation.	10
Figure 9 : merge synchronization in multiprocessing	10
Figure 10 : waitpid().	11
Figure 11 : multiprocessing with 2 processes output	11
Figure 12 :snippet code of the multithreading approch.	13
Figure 13 : thread structure.	14
Figure 14 : pthread_create	14
Figure 15 : The processthread function.	14
Figure 16 : merge function with mutex.	15
Figure 17 : pthread_join function,	15
Figure 18 :Multithreading sample output for 8 threads.	17

Theory

Naive approach

The naive approach to parallel computing refers to executing tasks in a sequential manner, where each task is handled one after the other within a single thread or process. This approach is easy to implement and can be effective for smaller or simpler tasks where the complexity of parallelization isn't justified. However, because tasks are performed serially, there is no speedup gained from parallelism. This method quickly becomes limiting for computationally intensive tasks that could significantly benefit from parallel execution, where dividing the work across multiple threads or processes would lead to faster and more efficient results.

It can be declared also that it is about diving into a task and trying to implement it right away, without overthinking the details. It's a quick and straightforward way to uncover any obstacles that might come up. By taking action, you'll usually discover any issues or dependencies through feedback from the compiler or testing, rather than trying to predict everything in advance.[1]

multiprocessing approach

A multiprocessing operating system is defined as a type of operating system that makes use of more than one CPU to improve performance. Multiple processors work paralleled in multi-processing operating systems to perform the given task. All the available processors are connected to peripheral devices, computer buses, physical memory, and clocks. The main aim of the multi-processing operating system is to increase the speed of execution of the system. [2]

Multi-threading approach

Multi-threading is the ability of a program or an operating system to enable more than one user at a time without requiring multiple copies of the program running on the computer. Multi-threading can also handle multiple requests from the same user.

Each user request for a program or system service is tracked as a thread with a separate identity. As programs work on behalf of the initial thread request and are interrupted by other requests, the work status of the initial request is tracked until the work is completed. In this context, a user can also be another program.

Fast central processing unit (CPU) speed and large memory capacities are needed for multi-threading. The single processor executes pieces, or threads, of various programs so fast, it appears the computer is handling multiple requests simultaneously.[3]

Discussion and code analysis

Common Code analysis

This part includes a simple C code without any parallelism or concurrency. The file is read all at once and stored in a one-dimensional array with memory allocated equal to the file size.

```
fseek(file, 0, SEEK_END);
long fileSize = ftell(file);
fseek(file, 0, SEEK_SET);

char *fileContent = (char *)malloc(fileSize + 1);
if (fileContent == NULL)
{
    printf("Memory allocation failed");
    fclose(file);
    return NULL;
}

fread(fileContent, 1, fileSize, file);
fileContent[fileSize] = '\0';
fclose(file);
return fileContent;
```

Figure 1:Read from file function.

Then, the word frequencies are counted in a simple way using an algorithm with an array of structs, where each struct is filled with the word and its frequency.

```
struct WordFreq
{
    char word[WORD_SIZE];
    int freq;
};

typedef struct WordFreq Word;
```

Figure 2 : Word-Freq struct.

Naive approach

Code

```
143 char *fileContent = OpenFileAndReadFromIt(fileName);
144 if (fileContent == NULL)
145 {
146     return 1;
147 }
148 int UniqueWords = 0;
149
150 Word *words = (Word *)malloc(MAX_WORDS * sizeof(struct WordFr
151
152 CountFreqOfTheWordsInTheFile(fileContent, words, &UniqueWords
153 HeapSort(words, UniqueWords);
154
155 printf("Top 10 Most Frequent Words:\n");
156 for (int i = UniqueWords - 1; i > (UniqueWords - 10); i--)
157 {
158     printf("%s: %d\n", words[i].word, words[i].freq);
159 }
160
161 free(fileContent);
162
```

Figure 3 : snippet code of the Naive.

Discussion and output analysis

I used a linear search to count the word frequencies by tokenizing each word in the file and iterating through the words array. If a word exists in the array, its frequency is incremented; if not, the word is added. The time complexity of the word frequency counting is $O(n^2)$ in general. After processing the entire file, the array is sorted using a min-heap sort, which has a time complexity of $O(n \log n)$. This is a simple, sequential implementation without any processing or threading. Since the algorithms used may not be the most efficient and the dataset is large, running the naive code produced the following results:

```
student@DESKTOP-R53LFAR:~/NewTry$ gcc 0sproject.c -o 0sproject
student@DESKTOP-R53LFAR:~/NewTry$ time ./0sproject
Top 10 Most Frequent Words:
1. the -> 1061396
2. of -> 593677
3. and -> 416629
4. one -> 411764
5. in -> 372201
6. a -> 325873
7. to -> 316376
8. zero -> 264975
9. nine -> 250430
10. two -> 192644

real    18m45.279s
```

Figure 4 : Naive results.

The code was run multiple times, and the execution time consistently ranged between 18 and 20 minutes (1060 to 1200 seconds), which demonstrates the performance of the approach through the following relation:

$$\text{Performance} = \frac{1}{\text{execution time}}$$

while the current approach doesn't make use of advanced techniques like parallel processing (which would allow tasks to run simultaneously), it still serves as a basic starting point. This basic approach can be improved upon in the future by using more advanced methods that would make the program run faster.

```
Top 10 Most Frequent Words:
the: 1061396
of: 593677
and: 416629
one: 411764
in: 372201
a: 325873
to: 316376
zero: 264975
nine: 250430
Execution time: 1222.288924 seconds
Wall-clock time: 1222.434081 seconds

real    20m22.438s
```

Figure 5: Naive execution time.

Multiprocessing Approach

Code

```
210 int numProcesses = MAX_PROCESSES;
211 int chunkSize = strlen(fileContent) / numProcesses;
212 pid_t pids[MAX_PROCESSES];
213
214 for (int i = 0; i < numProcesses; i++)
215 {
216     int startIdx = i * chunkSize;
217     int endIdx = (i == numProcesses - 1) ? strlen(fileContent) : startIdx + chunkSize;
218
219     while (!isspace(fileContent[endIdx]) && fileContent[endIdx] != '\n' && fileContent[endIdx] != '\0')
220     {
221         endIdx++;
222     }
223
224     if ((pids[i] = fork()) == 0)
225     {
226         Word *localWords = malloc(sizeof(Word) * MAX_WORDS);
227         int localWordCount = 0;
228         char *chunk = strndup(fileContent + startIdx, endIdx - startIdx);
229         CountFreqOfTheWordsInTheFile(chunk, localWords, &localWordCount, sharedWords, sharedWordCount, &lock);
230
231         free(chunk);
232         free(localWords);
233         exit(0);
234     }
235 }
236
237 for (int i = 0; i < numProcesses; i++)
238 {
239     waitpid(pids[i], NULL, 0);
240 }
```

Figure 6 : snippet code of multiprocessing.

Discussion and code analysis

Since the main idea of the approach is to create multiple processes and divide the data or the tasks between them to improve performance , in the snippet code , the data has been divided into small chunks equal the number of processes,that has been created using the fork() system call in a loop , each process has its own chunk and its own array to count frequencies locally and then merge the output of each process.

```
214 for (int i = 0; i < numProcesses; i++)
215 {
216     int startIdx = i * chunkSize;
217     int endIdx = (i == numProcesses - 1) ? strlen(fileContent) : startIdx + chunkSize;
218
219     while (!isspace(fileContent[endIdx]) && fileContent[endIdx] != '\n' && fileContent[endIdx] != '\0')
220     {
221         endIdx++;
222     }
223
224     if ((pids[i] = fork()) == 0)
225     {
226         Word *localWords = malloc(sizeof(Word) * MAX_WORDS);
227         int localWordCount = 0;
228         char *chunk = strndup(fileContent + startIdx, endIdx - startIdx);
229         CountFreqOfTheWordsInTheFile(chunk, localWords, &localWordCount, sharedWords, sharedWordCount, &lock);
230     }
```

Figure 7:dividing to chunks code.

To merge the outputs from multiple processes into one location, Inter-Process Communication (IPC) is required so that the processes can communicate and access each other's data. In this case, shared memory is used, which allows processes to share data directly.

```
sem_t lock;
sem_init(&lock, 1, 1);

Word *sharedWords = mmap(NULL, sizeof(Word) * MAX_WORDS, PROT_READ | PROT_WRITE, MAP_SHARED | MAP_ANONYMOUS, -1, 0);
if (sharedWords == MAP_FAILED)
{
    perror("mmap failed for sharedWords");
    free(fileContent);
    return 1;
}
```

Figure 8 : shared memory creation.

The shared memory is allocated using `mmap()` for an array of structure `Word` with memory space occupied by `MAX_WORDS` words. This memory type is readable and writable, as well as shared between processes or threads. It remains anonymous because memory is not linked to any file; it is simply a block of memory the operating system has created for itself.

```
if (*localWordCount >= 5000)
{
    sem_wait(lock);
    MergeResults(sharedWords, sharedWordCount, localWords, *localWordCount);
    sem_post(lock);
    *localWordCount = 0;
}
```

Figure 9 : merge synchronization in multiprocessing

To prevent processes from getting stuck for a long time in the critical section, the merging of results happens directly after each process processes a certain number of words. This ensures that no process holds the critical section for too long. To maintain data consistency and avoid race conditions, a binary semaphore is used. The binary semaphore ensures that only one process can access the critical section at a time, allowing safe merging of the data without conflicts between processes.

After each child process completes its work, it terminates by calling the `exit()` system call. To ensure that the parent process waits for all child processes to finish before continuing, the `waitpid()` function is used. This function allows the parent to wait for a specific child process to finish or wait for any child to terminate, ensuring proper synchronization and that the parent only proceeds once all child processes have completed their work.

```
5
6
7     for (int i = 0; i < numProcesses; i++)
8     {
9         waitpid(pids[i], NULL, 0);
10    }
```

Figure 10: `waitpid()`.

Performance evaluation involves measuring the execution time and throughput of the multiprocessing approach with varying numbers of processes to assess its impact on efficiency and scalability. By testing different configurations, we can determine how the number of processes influences overall performance, identifying the optimal setup for maximizing throughput. Throughput is calculated as the inverse of execution time ($\text{Performance} = 1 / \text{Time}$), where higher throughput indicates better performance, as it reflects the ability to complete tasks more quickly.

```
● student@DESKTOP-R53LFAR:~/NewTry$ time ./MultiProcessing
Top 10 Most Frequent Words:
the: 1061396
of: 593677
and: 416629
one: 411764
in: 372201
a: 325873
to: 316376
zero: 264975
nine: 250430
Execution time: 0.205779 seconds
Wall-clock time: 705.986224 seconds
```

Figure 11: multiprocessing with 2 processes output

Output execution time and performance for different number of processes :

processes	Execution time(Avg)(s)	Performance	Real time(S)
2	$\frac{0.2339+0.2495+0.2143}{3}$ =0.2325	4.531	653s
4	$\frac{0.2089+0.2057+0.2031}{3}$ = 0.2059	4.856	504s
6	$\frac{0.2266+0.2172+0.2047}{3}$ =0.2161	4.627	484s
8	$\frac{0.2325+0.2134+0.2162}{3}$ =0.2207	4.531	480s

Performance analysis

The number of processes used directly affects both execution time and overall efficiency. Increasing the number of processes generally speeds up task completion and improves throughput by taking advantage of parallelism, where tasks are divided and executed concurrently. However, having too many processes can create overhead, as they compete for system resources like CPU, memory, and I/O, which can actually slow down performance. Through experimentation on a system with 4 CPUs, using 4 processes was found to be the best strategy, as it matched the number of available CPU cores.

Multi-threading approach

Code

```
pthread_t threads[NUM_THREADS];
ThreadData threadData[NUM_THREADS];
Word *localWords[NUM_THREADS];
int localWordCounts[NUM_THREADS];

long fileSize = strlen(fileContent);
int chunkSize = fileSize / NUM_THREADS;
long endIdx = chunkSize;

for (int i = 0; i < NUM_THREADS; i++)
{
    localWords[i] = malloc(MAX_WORDS * sizeof(Word));
    localWordCounts[i] = 0;

    long startIdx = (i == 0) ? 0 : endIdx;
    endIdx = (i == NUM_THREADS - 1) ? fileSize : (startIdx + chunkSize);

    while (endIdx < fileSize && fileContent[endIdx] != ' ' && fileContent[endIdx] != '\n')
        endIdx--;

    threadData[i].fileChunk = strdup(fileContent + startIdx, endIdx - startIdx);
    threadData[i].localWords = localWords[i];
    threadData[i].localWordCount = &localWordCounts[i];
    threadData[i].mutex = &threadMutexes[i];
    threadData[i].ThreadNumber = i;

    pthread_create(&threads[i], NULL, ThreadProcess, &threadData[i]);
}

for (int i = 0; i < NUM_THREADS; i++)
{
    pthread_join(threads[i], NULL);
    pthread_mutex_t threadMutexes[NUM_THREADS];
    free(localWords[i]);
}
```

Figure 12: snippet code of the multithreading approach.

Discussion and code analysis

In this approach, the chunk division is similar to the multiprocessing method, but the key difference is that threads can communicate and share data directly without requiring IPC (Inter-Process Communication). Threads operate within the same memory space, allowing them to access each other's work seamlessly. The threads are created using the `pthread_t` type, which defines each thread. Additionally, a struct was created to store the data specific to each thread, enabling organized and efficient management of thread-related information.

```

25 typedef struct
26 {
27     char *fileChunk;
28     Word *localWords;
29     int *localWordCount;
30 }
31 ThreadData;
32

```

Figure 13 : thread structure.

The `pthread_create` function is used to create a new thread, assigning it a unique thread ID stored in `threads[i]`. The second argument is `NULL`, which specifies that the thread will use default attributes. The newly created thread executes the `ThreadProcess` function, which defines the work that the thread will perform. Additionally, `&threadData[i]` is passed to the thread as an argument, providing it with specific data to process. This allows each thread to work independently while sharing the same overall structure and functionality.

```

pthread_create(&threads[i], NULL, ThreadProcess, &threadData[i]);

```

Figure 14: pthread_create

```

void *ThreadProcess(void *arg)
{
    ThreadData *data = (ThreadData *)arg;
    *data->localWordCount = 0;
    ProcessFileChunk(data);
    MergeResults(data);
    free(data->fileChunk);
    return NULL;
}

```

Figure 15: The processthread function.

To prevent race conditions, where two threads try to update the shared data simultaneously, a mutex is used to lock the merge function. This ensures that only one

thread can perform the merge operation at a time, maintaining data consistency and preventing conflicts between threads during the merging process.

```
void MergeResults(ThreadData *data)
{
    pthread_mutex_lock(&globalMutex);

    for (int i = 0; i < *data->localWordCount; i++)
    {
        int found = 0;
        for (int j = 0; j < sharedWordCount; j++)
        {
            if (strcmp(sharedWords[j].word, data->localWords[i].word) == 0)
            {
                sharedWords[j].freq += data->localWords[i].freq;
                found = 1;
                break;
            }
        }

        if (!found && sharedWordCount < MAX_WORDS)
        {
            strcpy(sharedWords[sharedWordCount].word, data->localWords[i].word);
            sharedWords[sharedWordCount].freq = data->localWords[i].freq;
            sharedWordCount++;
        }
    }

    pthread_mutex_unlock(&globalMutex);
}
```

Figure 16: merge function with mutex.

To ensure that the main process waits for all threads to finish their work before continuing, the `pthread_join` function is used. This function blocks the main process until a specific thread has completed its execution, ensuring proper synchronization and preventing the main process from proceeding prematurely.

```
for (int i = 0; i < NUM_THREADS; i++)
{
    pthread_join(threads[i], NULL);
    free(localWords[i]);
}
```

Figure 17 : `pthread_join` function,

Performance and execution time analysis

Output execution time and performance for different number of processes :

threads	Execution time(Avg)	performance	Reel time
2	$\frac{1087.88+1060.62+1017.40}{3}$ =1055.3s	0.00094	411
4	$\frac{1116.92+1086.98+1065.04}{3}$ = 1089.6s	0.000951	448s
6	$\frac{1082.06+1317+1120.34}{3} =$ 1176.46s	0.00085	401
8	$\frac{926.06+999+1200.1}{3}$ =1063.72s	0.000963	383s

The differences in running time of a multithreaded approach to the wall-clock time result from the fact that running time refers to all the CPU times used by all threads combined, while wall-clock time represents the total real-world time taken by the program to run. All threads run concurrently, hence all their CPU times add to make a bigger running time. For instance, four threads running concurrently for about 100 seconds will use 100 actual wall-clock seconds, with 400 seconds counted for execution time.

That shows one of the gains of multithreaded approaches in terms of performance in parallelism. Wall-clock time is reduced, thanks to parallelism; however, the accumulated running time becomes larger, on account of the work done by the thread. There is also an increase in the accumulated execution time from overheads introduced by thread synchronization, resource contention, and context-switching costs.

```
Student@DESKTOP-R55LPAK:~/NewTty$ time ./thread
Top 10 most frequent words:
the: 1061396
of: 593677
and: 416629
one: 411764
in: 372201
a: 325873
to: 316376
zero: 264975
nine: 250430
two: 192644
Execution time: 992.715310 seconds
Wall-clock time: 411.164289 seconds

real    6m51.169s
```

Figure 18: Multithreading sample output for 8 threads.

Comparing performance of the approaches

approach	Number Of children	Execution time(Avg)	performance
Naive	-	1222	0.00081
processes	2 processes	0.2325	4.531
	4 processes	0.2059	4.856
	6 processes	0.2161	4.627
	8 processes	0.2207	4.531
Threads	2 Threads	1055.3	0.00094
	4 Threads	1089.6	0.000951
	6 Threads	1176.46	0.00085
	8 Threads	1063.72	0.000963

The Threads approach results in faster execution times compared to the Processes approach, possibly due to lower overhead for thread creation and management. However, the performance remains low across all thread counts, suggesting that while threads complete tasks faster, they may not be using system resources as effectively.

The Processes approach, while slightly slower than the threads in execution time, strikes a better balance between execution time and performance. It scales well and remains more efficient with fewer processes (optimal at 4 processes), making it a more reliable solution for tasks that require consistency and effective use of system resources.

The Naive approach remains the least efficient, with an execution time of 1222 seconds and poor performance. It serves as a baseline for comparison but shows that without parallel processing, the task completion time is significantly higher.

An analysis according to Amdahl's law.

A. Naive approach

What percentage is the serial part of your code?

Average Total Time: 1222sec.

Serial Time = Average Time for counting words frequencies: 1134.23 sec.

$$\text{Serial Portion} = \frac{\text{Serial Time}}{\text{Total Time}} \times 100\% = \frac{1134.23}{1222} \times 100\% = 92.8\%$$

$$\text{Maximum Speed up} = \frac{1}{s + \frac{(1-s)}{N}} = \frac{1}{92.8 + \frac{(1-92.8)}{1}} = 1.$$

B. Multiprocessing approach with 4 cores

Average Total Time: 520.3sec.

Parallel time = Average Time for counting words frequencies: 503 sec.

$$\text{Parallel Portion} = \frac{\text{Parallel Time}}{\text{Total Time}} \times 100\% = 96.7\%$$

$$S(n) = \frac{1}{(1-p) + \frac{p}{n}} = \frac{1}{3.3 + \frac{96.7}{4}} = 0.036$$

C. Multi-threading approach with 4 Threads

Average Total Time: 545.079068 seconds

Parallel time = Average Time for counting words frequencies: 448 sec.

$$\text{Parallel Portion} = \frac{\text{Parallel Time}}{\text{Total Time}} \times 100\% = 82.2\%$$

$$S(n) = \frac{1}{(1-p) + \frac{p}{n}} = \frac{1}{17.8 + \frac{82.2}{4}} = 0.02607$$

Questions analysis

1. Percentage of the Serial Part

The serial portion of the code is the part that cannot be parallelized. It is calculated as:

Naive Approach:

Serial Portion = 92.8%

This means that most of the code is sequential, leaving only 7.2% of the code parallelized.

Multiprocessing Approach (4 Cores):

Parallel Portion = 96.7%

Serial Portion = $1 - 0.967 = 0.033$ $0.033 * 100\% = 3.3\%$

Only a small portion of the code is sequential, making this approach highly parallelized.

Multi-threading Approach (4 Threads):

Parallel Portion = 82.2%

Serial Portion = $1 - 0.822 = 0.178$ $0.178 * 100\% = 17.8\%$

A larger portion of the code remains sequential compared to multiprocessing, reducing the benefits of parallelization.

2. Maximum Speedup According to the Available Number of Cores

Amdahl's Law calculates the **theoretical maximum speedup** for a given number of cores (n) as:

Naive Approach:

Serial Portion = 92.8%

$$\text{Maximum Speed up} = \frac{1}{s + \frac{(1-s)}{N}} = \frac{1}{92.8 + \frac{(1-92.8)}{1}} = 1.$$

Even with 4 cores, the maximum speedup is **1.06x** due to the large serial portion

Multiprocessing Approach (4 Cores):

With $P=0.967$:

$$\text{serial Portion} = 1 - P = 1 - 0.967 = 0.033$$

$$S(n) = \frac{1}{(1-p) + \frac{p}{n}} = \frac{1}{3.3 + \frac{96.7}{4}} = 0.036$$

The maximum speedup with 4 cores is approximately **3.64x**, which is close to ideal.

Multi-threading Approach (4 Threads):

With $P=0.822$

$$\text{serial Portion} = 1 - P = 1 - 0.822 = 0.178$$

$$S(n) = \frac{1}{(1-p) + \frac{p}{n}} = \frac{1}{17.8 + \frac{82.2}{4}} = 0.02607$$

The maximum speedup with 4 threads is **2.61x**, which is lower than multiprocessing due to higher synchronization and overhead costs.

3. Optimal Number of Child Processes or Threads

The best number of processes or threads typically matches the number of available cores. In this case, the system has 4 cores, so the optimal number is 4 processes for the multiprocessing approach or 4 threads for the multi-threading approach.

In conclusion, Multiprocessing shows the highest speedup and efficiency, making it the most effective approach for this task.

Work environment

I developed and executed my code on a physical computer with the following specifications:

- **CPU:** 4 cores, 2.7 GHz base clock speed.
- **Memory (RAM):** 8 GB.
- **Operating System:** Windows 10 running WSL 2 (Windows Subsystem for Linux) with Ubuntu 20.04 LTS as the Linux distribution.
- **Programming Language:** C.
- **IDE/Compiler:** Visual Studio Code for editing, with GCC 9.3.0 for compiling the C code inside WSL.
- I ran the code using WSL 2 on my Windows system, which provides a Linux-like environment for better compatibility with Linux-based tools.

Conclusion

In this project, we compared three methods for counting word frequencies: the naive sequential approach, multiprocessing, and multi-threading. The naive approach, while simple, was the least efficient because it doesn't take advantage of parallelism and had the longest execution time.

The multiprocessing approach, which uses separate processes to handle tasks in parallel, performed the best, reducing execution time significantly and providing the highest throughput. However, the multi-threading approach faced challenges like synchronization issues, which led to slower performance compared to multiprocessing.

These results emphasize the importance of choosing the right parallel computing method based on the specific task and available resources. Overall, this project offers valuable insights into how different approaches impact performance in word counting tasks and similar computations.

References

- [2]: [Multi Processing Operating System – GeeksforGeeks](#)
- [3]: [What is multithreading?](#)

